RouteAI — Systemanalys & Förbättringsförslag

> **Datum:** 2026-03-14
> **Systemstatus:** Funktionellt med 12 rutter, 24 stopp, 5 fordon, 4 förare
> **Övergripande bedömning:** Solid grund — systemet levererar redan värde. Förslagen nedan handlar om att ta det från "fungerar bra" till "produktionsredo och skalbart".

Innehåll

1. Arkitektur & Kodstruktur
2. Prestanda & Optimering
3. UX/UI
4. Funktionella Förbättringar
5. AI & Automation
6. Säkerhet & Multi-tenant
7. Hållbarhet
8. Replit-specifika Optimeringar
9. Implementeringsplan

---

1. Arkitektur & Kodstruktur

1.1 API-lagerstruktur

Prioritet	Förslag	Typ
🔴 Hög	Separera route handlers från affärslogik	Strategic
🟡 Medel	Inför ett service-layer mönster	Strategic
🟢 Låg	Centraliserad felhantering med error middleware	Quick win

Konkret: Service-layer separation

Istället för att ha all logik i route handlers, bryt ut till dedikerade services:

```
server/
├── routes/          # Enbart HTTP-hantering, validering, response
│   ├── routes.ts
│   ├── vehicles.ts
│   └── drivers.ts
├── services/        # Affärslogik
│   ├── routeOptimizer.ts
│   ├── vehicleService.ts
│   └── dispatchService.ts
├── repositories/    # Databasåtkomst (Drizzle-queries)
│   ├── routeRepo.ts
│   └── vehicleRepo.ts
└── middleware/
    ├── auth.ts
    ├── tenantResolver.ts
    └── errorHandler.ts
```

Varför: Gör koden testbar, återanvändbar och enklare att resonera kring. En route handler ska bara validera input, anropa en service och returnera response.

1.2 Drizzle ORM — Schema & Migrationer

Prioritet	Förslag	Typ
🔴 Hög	Inför `drizzle-kit` migrationer istället för `push`	Quick win
🟡 Medel	Lägg till database indexes på vanliga queries	Quick win
🟡 Medel	Typade relationer med Drizzle `relations()`	Strategic

Konkret: Index-strategi

```
// Dessa index bör finnas baserat på systemets query-mönster:
export const routes = pgTable('routes', {
  id: serial('id').primaryKey(),
  tenantId: integer('tenant_id').notNull(),
  status: varchar('status', { length: 50 }),
  scheduledDate: date('scheduled_date'),
  // ...
}, (table) => ({
  tenantIdx: index('idx_routes_tenant').on(table.tenantId),
  tenantDateIdx: index('idx_routes_tenant_date').on(table.tenantId, table.scheduledDat
e),
  statusIdx: index('idx_routes_status').on(table.status),
}));

export const stops = pgTable('stops', {
  id: serial('id').primaryKey(),
  routeId: integer('route_id').notNull(),
  lat: doublePrecision('lat'),
  lng: doublePrecision('lng'),
  // ...
}, (table) => ({
  routeIdx: index('idx_stops_route').on(table.routeId),
  locationIdx: index('idx_stops_location').on(table.lat, table.lng),
}));
```

1.3 Validering & Typning

Prioritet	Förslag	Typ
🔴 Hög	Zod-scheman för all API-input	Quick win
🟡 Medel	Delad typdefinition (shared types) mellan frontend/ backend	Strategic

Konkret:

```
// shared/schemas/route.ts — delas mellan frontend och backend
import { z } from 'zod';

export const createRouteSchema = z.object({
  name: z.string().min(1).max(200),
  vehicleId: z.number().int().positive(),
  driverId: z.number().int().positive().optional(),
  scheduledDate: z.string().datetime(),
  stops: z.array(z.object({
    address: z.string().min(1),
    lat: z.number().min(-90).max(90),
    lng: z.number().min(-180).max(180),
    timeWindowStart: z.string().datetime().optional(),
    timeWindowEnd: z.string().datetime().optional(),
  })).min(1),
});

export type CreateRouteInput = z.infer<typeof createRouteSchema>;
```

1.4 Miljöhantering

Prioritet	Förslag	Typ
🟡 Medel	Centraliserad config med validering	Quick win

```
// server/config.ts
import { z } from 'zod';

const envSchema = z.object({
  DATABASE_URL: z.string().url(),
  OPENAI_API_KEY: z.string().min(1),
  OSRM_BASE_URL: z.string().url().default('http://router.project-osrm.org'),
  SESSION_SECRET: z.string().min(32),
  NODE_ENV: z.enum(['development', 'production']).default('development'),
  MAX_STOPS_PER_ROUTE: z.coerce.number().default(100),
});

export const config = envSchema.parse(process.env);
// Kraschar tidigt vid startup om config saknas — mycket bättre än runtime-errors
```

---

2. Prestanda & Optimering

2.1 Databasoptimering

Prioritet	Förslag	Typ
🔴 Hög	Connection pooling med korrekt pool-storlek	Quick win
🔴 Hög	Paginering på alla list-endpoints	Quick win
🟡 Medel	Materialized views för dashboard-data	Strategic

Konkret: Paginering

```
// Cursor-baserad paginering – stabilare än OFFSET vid stora dataset
async function getRoutes(tenantId: number, cursor?: number, limit = 25) {
  const query = db
    .select()
    .from(routes)
    .where(
      and(
        eq(routes.tenantId, tenantId),
        cursor ? gt(routes.id, cursor) : undefined,
      )
    )
    .orderBy(asc(routes.id))
    .limit(limit + 1); // +1 för att veta om det finns fler

  const results = await query;
  const hasMore = results.length > limit;
  return {
    data: results.slice(0, limit),
    nextCursor: hasMore ? results[limit - 1].id : null,
  };
}
```

Konkret: Dashboard Materialized View

```
-- Snabb dashboard utan att räkna om varje gång
CREATE MATERIALIZED VIEW dashboard_stats AS
SELECT
  tenant_id,
  COUNT(DISTINCT r.id) as total_routes,
  COUNT(DISTINCT s.id) as total_stops,
  COUNT(DISTINCT r.id) FILTER (WHERE r.status = 'active') as active_routes,
  SUM(r.total_distance_km) as total_km,
  SUM(r.co2_saved_kg) as total_co2_saved,
  DATE_TRUNC('day', NOW()) as computed_at
FROM routes r
LEFT JOIN stops s ON s.route_id = r.id
GROUP BY tenant_id;

-- Refresh med cron eller efter bulk-operationer
REFRESH MATERIALIZED VIEW CONCURRENTLY dashboard_stats;
```

2.2 Kartrendering (Leaflet)

Prioritet	Förslag	Typ
🔴 Hög	Marker clustering vid >50 stopp	Quick win
🟡 Medel	Lazy loading av rutter som inte syns	Quick win
🟡 Medel	Canvas-renderer istället för SVG vid många element	Strategic

Konkret: Marker Clustering

```
import MarkerClusterGroup from 'react-leaflet-cluster';

function RouteMap({ stops }: { stops: Stop[] }) {
  return (
    <MapContainer center={[59.33, 18.07]} zoom={12}>
      <TileLayer url="https://placehold.co/1200x600/e2e8f0/1e293b?
text=A_map_tile_image_from_OpenStreetMap_depicting_deta" />
      <MarkerClusterGroup
        chunkedLoading
        maxClusterRadius={50}
        spiderfyOnMaxZoom
        showCoverageOnHover={false}
      >
        {stops.map(stop => (
          <Marker key={stop.id} position={[stop.lat, stop.lng]}>
            <Popup>{stop.address}</Popup>
          </Marker>
        ))}
      </MarkerClusterGroup>
    </MapContainer>
  );
}
```

2.3 Realtidsuppdateringar

Prioritet	Förslag	Typ
🟡 Medel	WebSocket med rum per ten-ant	Strategic
🟡 Medel	Debounce position-updates (max 1/5s per fordon)	Quick win
🟢 Låg	Server-Sent Events fallback	Quick win

Konkret: WebSocket-rum

```
// server/websocket.ts
import { Server } from 'socket.io';

export function setupWebSocket(server: HttpServer) {
  const io = new Server(server, { cors: { origin: '*' } });

  io.use(authenticateSocket); // Verifiera JWT/session

  io.on('connection', (socket) => {
    const tenantId = socket.data.tenantId;
    socket.join(`tenant:${tenantId}`);

    // Skicka bara uppdateringar till rätt tenant
    socket.on('vehicle:position', (data) => {
      io.to(`tenant:${tenantId}`).emit('vehicle:moved', {
        vehicleId: data.vehicleId,
        lat: data.lat,
        lng: data.lng,
        timestamp: Date.now(),
      });
    });
  });

  return io;
}
```

2.4 Frontend-prestanda

Prioritet	Förslag	Typ
🔴 Hög	React.memo + useMemo på kartkomponenter	Quick win
🟡 Medel	Code splitting per route (React.lazy)	Quick win
🟡 Medel	Virtualiserad lista för stopptabeller (TanStack Virtual)	Strategic

```
// Lazy-loading av tunga sidor
const MapView = React.lazy(() => import('./pages/MapView'));
const GanttTimeline = React.lazy(() => import('./pages/GanttTimeline'));
const Analytics = React.lazy(() => import('./pages/Analytics'));

function App() {
  return (
    <Suspense fallback={<PageSkeleton />}>
      <Routes>
        <Route path="/map" element={<MapView />} />
        <Route path="/dispatch" element={<GanttTimeline />} />
        <Route path="/analytics" element={<Analytics />} />
      </Routes>
    </Suspense>
  );
}
```

3. UX/UI

3.1 Navigering & Arbetsflöden

Prioritet	Förslag	Typ
🔴 Hög	Guided onboarding-wizard för nya användare	Strategic
🔴 Hög	Keyboard shortcuts för vanliga operationer	Quick win
🟡 Medel	Breadcrumbs + kontextuell navigation	Quick win
🟡 Medel	Bulk-operationer (välj flera rutter → ändra status)	Strategic

Konkret: Keyboard Shortcuts

```
// hooks/useKeyboardShortcuts.ts
import { useHotkeys } from 'react-hotkeys-hook';

export function useGlobalShortcuts() {
  useHotkeys('ctrl+n', () => navigate('/routes/new'), { description: 'Ny rutt' });
  useHotkeys('ctrl+m', () => navigate('/map'), { description: 'Kartvy' });
  useHotkeys('ctrl+d', () => navigate('/dispatch'), { description: 'Dispatch' });
  useHotkeys('ctrl+k', () => openCommandPalette(), { description: 'Sök' });
  useHotkeys('?', () => showShortcutsDialog(), { description: 'Visa genvägar' });
}
```

Konkret: Command Palette (Ctrl+K)

En “Spotlight”-liknande sökruta som låter användaren snabbt navigera, söka rutter, fordon, förare eller utföra åtgärder. Extremt effektiv UX-förbättring.

```
// Sökresultat-typer
type CommandResult =
  | { type: 'route'; id: number; name: string }
  | { type: 'vehicle'; id: number; plate: string }
  | { type: 'driver'; id: number; name: string }
  | { type: 'action'; label: string; fn: () => void };
```

3.2 Kartinteraktion

Prioritet	Förslag	Typ
🔴 Hög	Drag-and-drop stopp-ordning direkt på kartan	Strategic
🟡 Medel	Split-vy: karta + stoppslista sida vid sida	Quick win
🟡 Medel	Färgkodade rutter per status (aktiv/klar/försenad)	Quick win

Konkret: Färgkodning

```
const ROUTE_COLORS: Record<RouteStatus, string> = {
  planned:   '#6B7280', // grå
  active:    '#3B82F6', // blå
  delayed:   '#F59E0B', // gul/orange
  completed: '#10B981', // grön
  cancelled: '#EF4444', // röd
};

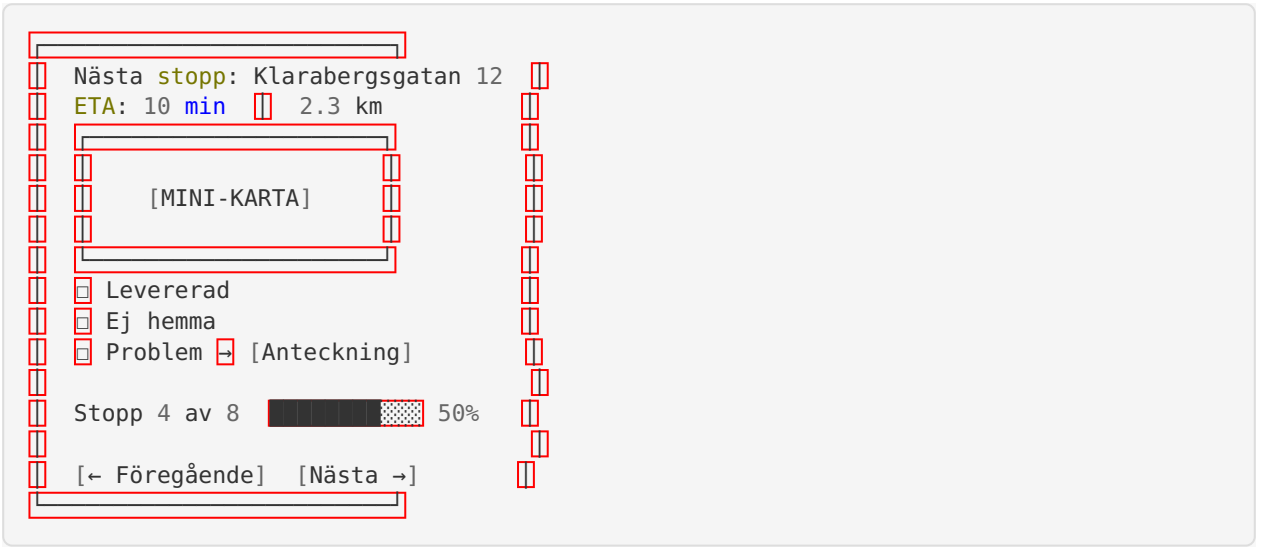
// Polyline med statusfärg
<Polyline
  positions={route.geometry}
  color={ROUTE_COLORS[route.status]}
  weight={route.status === 'active' ? 5 : 3}
  opacity={route.status === 'cancelled' ? 0.4 : 0.9}
/>
```

3.3 Responsivitet & Tillgänglighet

Prioritet	Förslag	Typ
🔴 Hög	ARIA-labels på alla interaktiva element	Quick win
🟡 Medel	Mobilvänlig vy för förare i fält	Strategic
🟡 Medel	High contrast-läge	Quick win
🟢 Låg	Skärmläsarstöd för kartan (textuell ruttbeskrivning)	Strategic

Konkret: Förar-mobilvy

Förare behöver en enkel, fokuserad vy — inte hela dispatch-panelen:



3.4 Feedback & Notiser

Prioritet	Förslag	Typ
🟡 Medel	Toast-notiser vid lyckade/ misslyckade operationer	Quick win
🟡 Medel	Optimistisk UI-uppdatering (uppdatera UI direkt, rollback vid fel)	Strategic
🟢 Låg	Ljud-notis vid geofence-event	Quick win

---

4. Funktionella Förbättringar

4.1 Ruttoptimering

Prioritet	Förslag	Typ
🔴 Hög	Tidsfönster-constraints i Clarke-Wright	Strategic
🔴 Hög	Kapacitetsbegränsningar (vikt/volym) per fordon	Strategic
🟡 Medel	Multi-depot-stöd	Strategic
🟡 Medel	Dynamisk omplanering vid förseningar	Strategic
🟢 Låg	Alternativa ruttförslag (snab- bast / kortast / grönast)	Strategic

Konkret: Tidsfönster

```
interface Stop {
  id: number;
  lat: number;
  lng: number;
  // Nytt: tidsfönster
  timeWindowStart?: Date; // "Leverera tidigast kl 09:00"
  timeWindowEnd?: Date;   // "Leverera senast kl 12:00"
  serviceDuration: number; // Minuter på plats (lastning, etc.)
}

// I Clarke-Wright: penalisera eller förkasta merges som bryter tidsfönster
function isTimeWindowFeasible(route: Stop[], travelTimes: number[][]): boolean {
  let currentTime = route[0].departureTime;
  for (let i = 1; i < route.length; i++) {
    currentTime += travelTimes[route[i-1].id][route[i].id];
    if (route[i].timeWindowEnd && currentTime > route[i].timeWindowEnd) {
      return false; // Kan inte leverera i tid
    }
    // Vänta om vi anländer för tidigt
    if (route[i].timeWindowStart && currentTime < route[i].timeWindowStart) {
      currentTime = route[i].timeWindowStart;
    }
    currentTime += route[i].serviceDuration;
  }
  return true;
}
```

4.2 Import & Export

Prioritet	Förslag	Typ
🔴 Hög	Validering med tydliga felmeddelanden vid import	Quick win
🟡 Medel	Mallnedladdning (Excel-mall med exempeldata)	Quick win
🟡 Medel	Export till PDF (ruttblad för förare)	Strategic
🟢 Låg	API-import (webhook för ordersystem)	Strategic

Konkret: Validering med feedback

```
interface ImportValidation {
  row: number;
  field: string;
  value: string;
  error: 'missing' | 'invalid_format' | 'out_of_range' | 'duplicate';
  suggestion?: string;
}

// Resultat visas som en tabell med rad-för-rad status:
// ✅ Rad 1: Klarabergsgatan 12 — OK
// ✅ Rad 2: Vasagatan 7 — OK
// ❌ Rad 3: Kolumn "postnummer" saknas
// ⚠️ Rad 4: Koordinater utanför Sverige — kontrollera
```

4.3 Geofencing

Prioritet	Förslag	Typ
🟡 Medel	Anpassningsbara geofence-radier per stopp	Quick win
🟡 Medel	Automatisk statusuppdatering vid entry/exit	Strategic
🟢 Låg	Historik-logg över geofence-events	Quick win

4.4 Kundspårning

Prioritet	Förslag	Typ
🔴 Hög	SMS/e-post-notis “Föraren är 10 min bort”	Strategic
🟡 Medel	Betygsättning efter leverans	Quick win
🟡 Medel	Anpassningsbar spårningssida (företagslogga, färger)	Quick win

---

5. AI & Automation

5.1 Prediktiv Analys

Prioritet	Förslag	Typ
🔴 Hög	ETA-prediktion baserad på historisk data	Strategic
🟡 Medel	Förseningsrisk-score per rutt	Strategic
🟢 Låg	Demand forecasting — förutse leveransvolymer	Strategic

Konkret: ETA-modell

```
// Samla in features för varje historisk leverans
interface DeliveryFeatures {
  dayOfWeek: number;
  hourOfDay: number;
  distanceKm: number;
  numStopsBefore: number;     // Hur många stopp innan detta
  isUrbanArea: boolean;
  weatherCondition: string;   // Från väder-API
  historicalAvgMinutes: number; // Snitt för denna sträcka
}

// Enkel approach: Använd OpenAI function calling för att
// analysera historisk data och ge ETA-uppskattning
async function predictETA(route: Route, currentStop: number): Promise<number> {
  const features = extractFeatures(route, currentStop);
  const historicalData = await getHistoricalDeliveries(route.tenantId, {
    similarRoutes: true,
    dayOfWeek: features.dayOfWeek,
    limit: 100,
  });

  // Viktad genomsnitt baserat på likhet
  return weightedAverage(historicalData, features);
}
```

5.2 AI-assistent Förbättringar

Prioritet	Förslag	Typ
🔴 Hög	Function calling — assistenten kan utföra åtgärder	Strategic
🟡 Medel	Kontext-medveten: assistenten “ser” aktuell vy	Quick win
🟡 Medel	Naturligt språk → ruttoptimering	Strategic

Konkret: Function Calling

```
const tools = [
  {
    type: 'function',
    function: {
      name: 'optimize_route',
      description: 'Optimera en befintlig rutt eller skapa en ny optimerad rutt',
      parameters: {
        type: 'object',
        properties: {
          routeId: { type: 'number', description: 'Rutt-ID att optimera' },
          constraints: {
            type: 'object',
            properties: {
              maxDuration: { type: 'number', description: 'Max tid i minuter' },
              prioritizeEV: { type: 'boolean', description: 'Prioritera elfordon' },
            }
          }
        }
      }
    }
  },
  {
    type: 'function',
    function: {
      name: 'get_route_summary',
      description: 'Hämta sammanfattning av en eller alla rutter',
      parameters: {
        type: 'object',
        properties: {
          routeId: { type: 'number' },
          dateRange: { type: 'string', enum: ['today', 'this_week', 'this_month'] },
        }
      }
    }
  },
  {
    type: 'function',
    function: {
      name: 'reassign_driver',
      description: 'Omtilldela en förare till en annan rutt',
      parameters: {
        type: 'object',
        properties: {
          routeId: { type: 'number' },
          driverId: { type: 'number' },
        },
        required: ['routeId', 'driverId']
      }
    }
  }
];

// Användaren kan nu säga:
// "Optimera rutt 5 med max 4 timmars körtid"
// "Vem kör minst idag? Flytta rutt 3 till den föraren."
// "Ge mig en sammanfattning av veckans leveranser."
```

5.3 Automatisering

Prioritet	Förslag	Typ
🟡 Medel	Auto-dispatch: tilldela rutter baserat på regler	Strategic
🟡 Medel	Smart notifiering vid avvikelser	Quick win
🟢 Låg	Automatisk rapportgenerering (daglig/veckovis)	Strategic

---

6. Säkerhet & Multi-tenant

6.1 Dataisolering

Prioritet	Förslag	Typ
🔴 Hög	Row Level Security (RLS) i PostgreSQL	Strategic
🔴 Hög	Tenant-ID validering på alla endpoints (middleware)	Quick win
🟡 Medel	Audit log för känsliga operationer	Strategic

Konkret: Row Level Security

```
-- Aktivera RLS på alla tenant-tabeller
ALTER TABLE routes ENABLE ROW LEVEL SECURITY;
ALTER TABLE stops ENABLE ROW LEVEL SECURITY;
ALTER TABLE vehicles ENABLE ROW LEVEL SECURITY;
ALTER TABLE drivers ENABLE ROW LEVEL SECURITY;

-- Policy: användare ser bara sin tenant's data
CREATE POLICY tenant_isolation ON routes
  USING (tenant_id = current_setting('app.current_tenant_id')::int);

CREATE POLICY tenant_isolation ON stops
  USING (route_id IN (
    SELECT id FROM routes WHERE tenant_id =
current_setting('app.current_tenant_id')::int
  ));
```

```
// middleware/tenantContext.ts — sätts vid varje request
async function setTenantContext(req: Request, res: Response, next: NextFunction) {
  const tenantId = req.user?.tenantId;
  if (!tenantId) return res.status(403).json({ error: 'No tenant context' });

  // Sätter PostgreSQL session-variabel för RLS
  await db.execute(sql`SELECT set_config('app.current_tenant_id', ${tenant-
Id.toString()}, true)`);
  next();
}
```

6.2 Autentisering & Auktorisering

Prioritet	Förslag	Typ
🔴 Hög	RBAC: Admin / Dispatcher / Förare / Viewer	Strategic
🟡 Medel	Rate limiting per tenant	Quick win
🟡 Medel	Session rotation vid privilegieändringar	Quick win
🟢 Låg	SSO-stöd (SAML/OIDC) för företagskunder	Strategic

Konkret: RBAC-modell

```
enum Role {
  ADMIN = 'admin',           // Full åtkomst
  DISPATCHER = 'dispatcher', // Rutter, stopp, dispatch — ej inställningar
  DRIVER = 'driver',         // Bara sin egen rutt, statusuppdatering
  VIEWER = 'viewer',         // Läsåtkomst till dashboard
}

const permissions: Record<Role, string[]> = {
  admin:      ['*'],
  dispatcher: ['routes:*', 'stops:*', 'vehicles:read', 'drivers:read', 'dispatch:*'],
  driver:     ['routes:read:own', 'stops:update:own', 'position:update'],
  viewer:     ['routes:read', 'dashboard:read', 'analytics:read'],
};

// Middleware
function requirePermission(permission: string) {
  return (req: Request, res: Response, next: NextFunction) => {
    const userPerms = permissions[req.user.role];
    if (userPerms.includes('*') || userPerms.includes(permission)) {
      return next();
    }
    return res.status(403).json({ error: 'Otillräckliga rättigheter' });
  };
}

// Användning
app.delete('/api/routes/:id', requirePermission('routes:delete'), deleteRoute);
```

6.3 API-säkerhet

Prioritet	Förslag	Typ
🔴 Hög	Input-sanitering (XSS, SQL injection via parameterisering)	Quick win
🟡 Medel	CORS — restriktiv origin-policy	Quick win
🟡 Medel	Helmet.js för säkerhets-headers	Quick win
🟢 Låg	API-nyckel-auth för externa integrationer	Strategic

```
// Quick win: 3 rader
import helmet from 'helmet';
import rateLimit from 'express-rate-limit';

app.use(helmet());
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100, keyGenerator: (req) => req.use
r?.tenantId }));
```

---

7. Hållbarhet

7.1 CO_2-beräkning

Prioritet	Förslag	Typ
🔴 Hög	Differentierad CO_2-beräkning per fordonstyp	Quick win
🟡 Medel	Jämförelse: optimerad rutt vs. naiv rutt	Quick win
🟡 Medel	Well-to-wheel beräkning för elfordon	Strategic

Konkret: Fordonsspecifik CO_2-modell

```typescript
interface VehicleEmissionProfile {
  type: 'diesel' | 'petrol' | 'electric' | 'hybrid' | 'biogas';
  co2PerKm: number;        // g CO₂/km (tailpipe)
  co2PerKmWTW: number;     // g CO₂/km (well-to-wheel, inkl. elproduktion)
  fuelConsumption: number;  // l/100km eller kWh/100km
  loadFactor: number;       // Extra förbrukning per ton last
}

const defaultProfiles: Record<string, VehicleEmissionProfile> = {
  diesel_van:    { type: 'diesel',   co2PerKm: 210, co2PerKmWTW: 250,
fuelConsumption: 8.5, loadFactor: 0.03 },
  diesel_truck:  { type: 'diesel',   co2PerKm: 350, co2PerKmWTW: 420,
fuelConsumption: 25,  loadFactor: 0.05 },
  electric_van:  { type: 'electric', co2PerKm: 0,   co2PerKmWTW: 15,
fuelConsumption: 20,  loadFactor: 0.02 },
  hybrid_van:    { type: 'hybrid',   co2PerKm: 120, co2PerKmWTW: 155,
fuelConsumption: 5.5, loadFactor: 0.025 },
};

function calculateRouteEmissions(route: Route, vehicle: Vehicle): EmissionResult {
  const profile = vehicle.emissionProfile ?? defaultProfiles[vehicle.type];
  const distanceKm = route.totalDistanceKm;
  const loadTons = route.totalLoadKg / 1000;

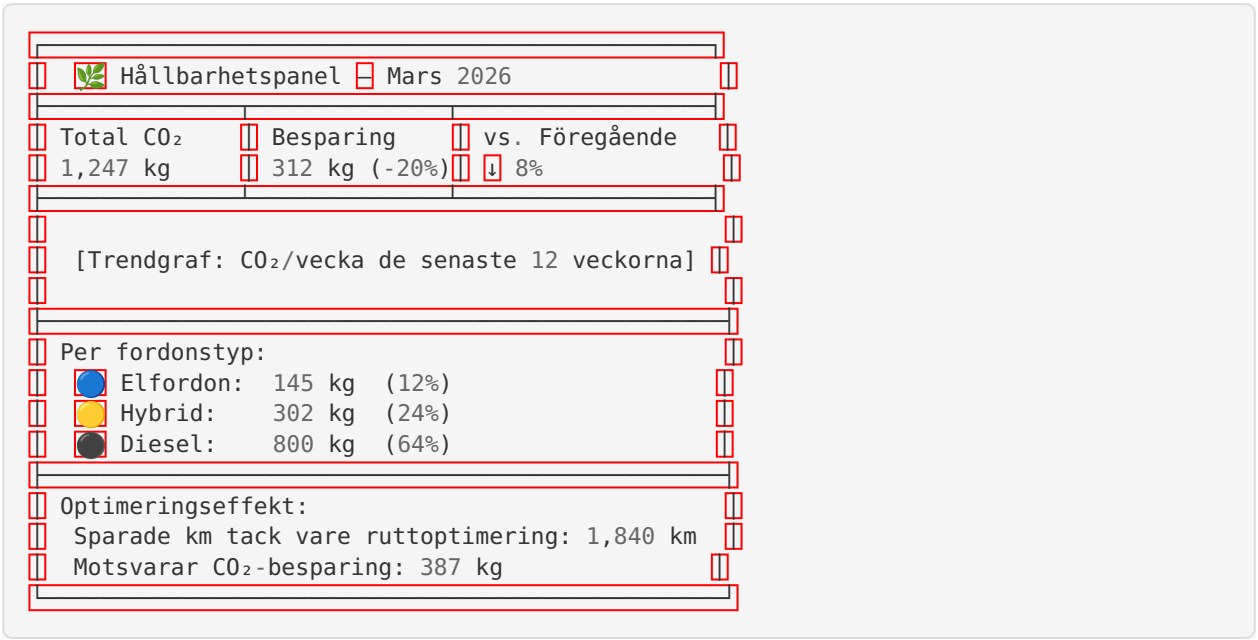
  const adjustedCO2 = profile.co2PerKmWTW * (1 + profile.loadFactor * loadTons);

  return {
    totalCO2Kg: (adjustedCO2 * distanceKm) / 1000,
    co2PerStop: (adjustedCO2 * distanceKm) / 1000 / route.stops.length,
    comparedToNaive: calculateNaiveCO2(route) - (adjustedCO2 * distanceKm) / 1000,
    fuelUsed: (profile.fuelConsumption * distanceKm) / 100,
  };
}
```

7.2 Rapportering

Prioritet	Förslag	Typ
🔴 Hög	Månatlig hållbarhetsrapport (PDF)	Strategic
🟡 Medel	Trendgraf: CO_2 per månad/vecka	Quick win
🟡 Medel	Jämförelse mot branschsnitt	Strategic
🟢 Låg	Export för GHG Protocol-rapportering	Strategic

Konkret: Dashboard-data



7.3 Elfordon-specifikt

Prioritet	Förslag	Typ
🟡 Medel	Batterinivå-tracking i realtid	Strategic
🟡 Medel	Optimala laddningstider (elpris-medveten)	Strategic
🟢 Låg	Laddstation-tillgänglighet via API (t.ex. Open Charge Map)	Strategic

---

8. Replit-specifika Optimeringar

8.1 Resursbegränsningar

Prioritet	Förslag	Typ
🔴 Hög	Effektiv minneshantering — undvik memory leaks	Quick win
🔴 Hög	Begränsa concurrency vid tunga beräkningar	Quick win
🟡 Medel	Cachning med node-cache (in-memory, ej Redis)	Quick win

Konkret: Enkel in-memory cache

```
import NodeCache from 'node-cache';

// Replit har inte Redis – men in-memory cache fungerar bra
// vid single-instance deployment
const cache = new NodeCache({
  stdTTL: 300,        // 5 minuter default
  checkperiod: 60,
  maxKeys: 1000,      // Begränsa minnesanvändning
});

// Användning
async function getRouteWithCache(routeId: number): Promise<Route> {
  const cacheKey = `route:${routeId}`;
  const cached = cache.get<Route>(cacheKey);
  if (cached) return cached;

  const route = await routeRepo.findById(routeId);
  cache.set(cacheKey, route);
  return route;
}

// Invalidera vid uppdatering
async function updateRoute(routeId: number, data: Partial<Route>) {
  const updated = await routeRepo.update(routeId, data);
  cache.del(`route:${routeId}`);
  cache.del(`dashboard:${updated.tenantId}`); // Invalidera dashboard-cache också
  return updated;
}
```

8.2 Cold Start & Uptime

Prioritet	Förslag	Typ
🔴 Hög	Always-on (Replit Reserved VM) om budget tillåter	Quick win
🟡 Medel	Graceful startup: lazy-load tunga moduler	Quick win
🟡 Medel	Health check endpoint för monitoring	Quick win

```
// Health check – viktig för Replit's uptime monitoring
app.get('/health', async (req, res) => {
  try {
    await db.execute(sql`SELECT 1`);
    res.json({
      status: 'ok',
      uptime: process.uptime(),
      memory: process.memoryUsage().rss / 1024 / 1024, // MB
      timestamp: new Date().toISOString(),
    });
  } catch {
    res.status(503).json({ status: 'unhealthy', error: 'database connection failed' })
;
  }
});
```

8.3 Replit PostgreSQL

Prioritet	Förslag	Typ
🔴 Hög	Automatisk backup-strategi (pg_dump cron)	Quick win
🟡 Medel	Connection pool size anpassad till Replit (max 5-10)	Quick win
🟢 Låg	Överväg Neon/Supabase för skalbarhet vid tillväxt	Strategic

```
// Replit PostgreSQL har begränsade connections
import { drizzle } from 'drizzle-orm/node-postgres';
import { Pool } from 'pg';

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 8,                    // Replit: håll lågt
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 5000,
});

export const db = drizzle(pool);
```

8.4 Deploymentstrategi

Prioritet	Förslag	Typ
🟡 Medel	Använd Replit Deployments (production mode)	Quick win
🟡 Medel	Separata dev/staging/prod miljöer via Replit forks	Strategic
🟢 Låg	Exitstrategi: Docker-beredskap för framtida migration	Strategic

Konkret: Dockerfile för framtida migration

```
# Redo om ni vill flytta från Replit i framtiden
FROM node:20-slim AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-slim
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json .
EXPOSE 3000
CMD ["node", "dist/server/index.js"]
```

---

9. Implementeringsplan

Fas 1: Quick Wins (Vecka 1–2)

Dessa kräver minimal refaktorering och ger omedelbar effekt:

#	Åtgärd	Uppskattad tid
1	Helmet.js + rate limiting	30 min
2	Zod-validering på top 5 end-points	2–3 tim
3	Database indexes	1 tim
4	Marker clustering på kartan	1–2 tim
5	Health check endpoint	30 min
6	Centraliserad env-config med validering	1 tim
7	Färgkodade rutter på kartan	1 tim
8	Keyboard shortcuts (Ctrl+K)	2–3 tim
9	CO_2-beräkning per fordonstyp	2 tim
10	Toast-notiser	1 tim

Total: ~2 arbetsdagar

Fas 2: Strukturella förbättringar (Vecka 3–6)

#	Åtgärd	Uppskattad tid
1	Service-layer separation	3–5 dagar
2	RBAC-implementation	2–3 dagar
3	Tenant middleware + RLS	2–3 dagar
4	WebSocket-baserad realtid	2–3 dagar
5	Tidsfönster i Clarke-Wright	2–3 dagar
6	Import-validering med felrap-port	1–2 dagar

Fas 3: Strategiska features (Vecka 7–12+)

#	Åtgärd	Uppskattad tid
1	AI function calling i assistenten	1 vecka
2	Förar-mobilvy	1–2 veckor
3	Prediktiv ETA-modell	1–2 veckor
4	Hållbarhetsrapport (PDF-export)	1 vecka
5	Kundnotifiering (SMS/e-post)	1 vecka
6	Auto-dispatch med regelmotor	2 veckor

Sammanfattning

RouteAI har en solid grund. Systemet levererar redan värde med fungerande ruttoptimering, kartvy, dispatch och CO_2-spårning. Förbättringarna ovan handlar inte om att "fixa" ett trasigt system — utan om att ta det vidare:

Top 5 rekommendationer i prioritetsordning:

1. **Säkerhetsgrund** — Helmet, rate limiting, tenant-middleware, RBAC (kritiskt för multi-tenant)
2. **Quick wins i UX** — Marker clustering, färgkodade rutter, keyboard shortcuts (omedelbar användarnytta)
3. **Service-layer** — Separera logik från routes (grunden för all framtida utveckling)
4. **AI function calling** — Gör assistenten handlingskraftig, inte bara rådgivande (wow-faktor)
5. **Hållbarhetsrapportering** — Differentierad CO_2 + PDF-rapport (affärsvärde, kundkrav)

> 💡 **Generell princip:** Gör inte allt samtidigt. Välj 2–3 quick wins per sprint, paketerat med en strukturell förbättring. Leverera inkrementellt — det håller motivationen uppe och minskar risken.

Rapporten genererad 2026-03-14. Baserad på systembeskrivning — rekommendationerna kan behöva justeras efter kodgranskning.